Computer Vision Project

Github Repository

Summary

- Project objective:
- Calendar
- Sprints
  - Sprint #1
  - Sprint #2
  - Sprint #3

Project objective:

`How to improve skin-disease detection in dark skin-tones using CNNs.`

Calendar

Date	Name	Objective
07/04	Sprint #1	Basic project structure and setting up the database. First tests with simple detection CNN
14/04	Sprint #2	Implementing segmentation to improve classification performance
28/04	Sprint #3	Creating web interface to use with the model
05/05	Sprint #4	Connecting web interface with the model and polishing last details
12/05	Sprint #5	Polishing phase

Sprints

Sprint #1

Resources needed: AWS resources to host the model backend.

Sprint #2

The model couldn't be trained on a personal notebook with 16 GB of ram. The estimated size required to train the entire database of images is around 40 GB. A smaller version was tested with a total of 500 input images and it required 8 GB to be loaded.

More work will be necessary in the following week in order to get the model working.

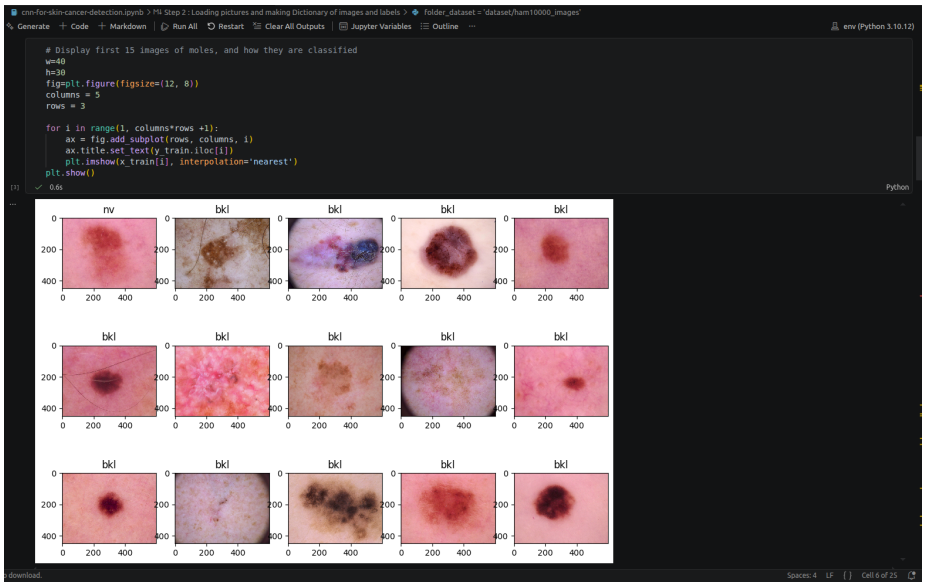
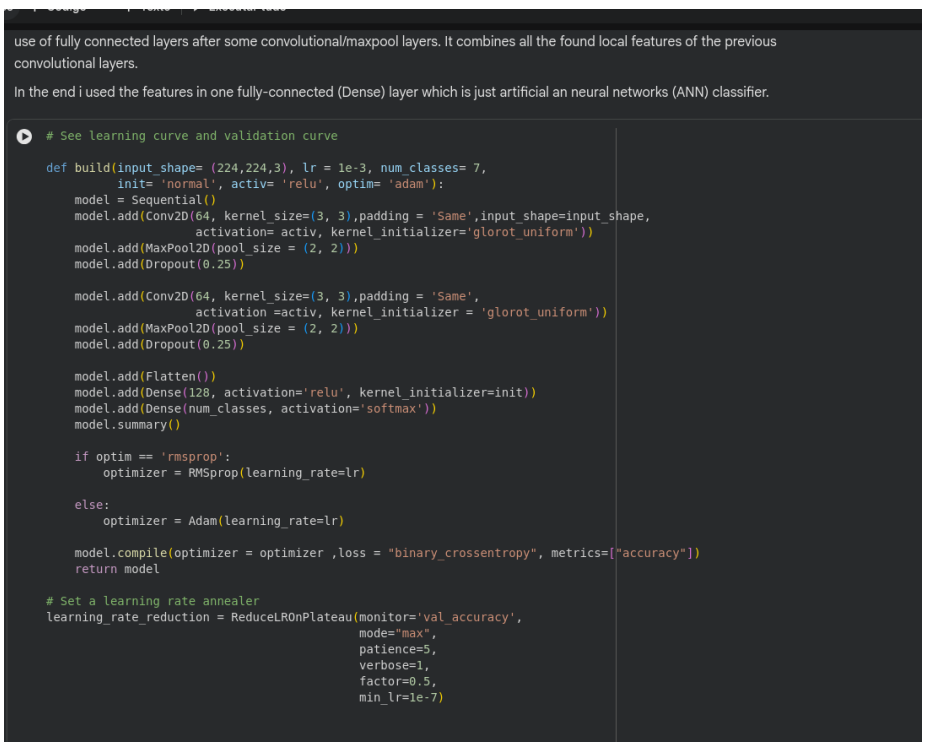


Figure 1: “Loading the database”



use of fully connected layers after some convolutional/maxpool layers. It combines all the found local features of the previous convolutional layers.

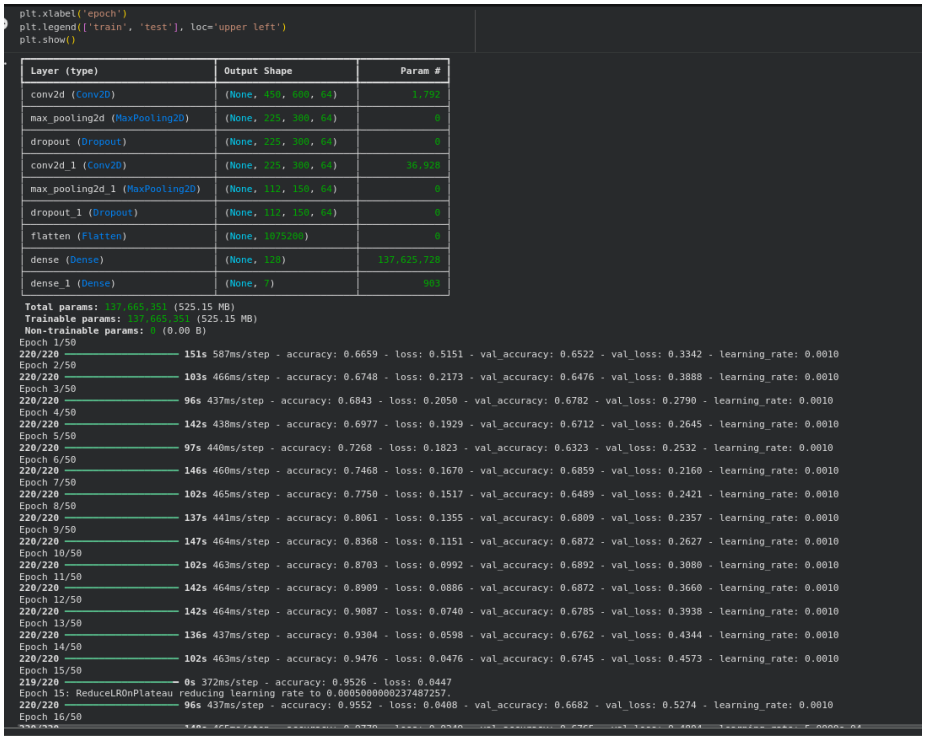
In the end i used the features in one fully-connected (Dense) layer which is just artificial an neural networks (ANN) classifier.

```
# See learning curve and validation curve

def build(input_shape= (224,224,3), lr = 1e-3, num_classes= 7,
          init= 'normal', activ= 'relu', optim= 'adam'):
    model = Sequential()
    model.add(Conv2D(64, kernel_size=(3, 3),padding = 'Same',input_shape=input_shape,
                     activation= activ, kernel_initializer='glorot_uniform'))
    model.add(MaxPool2D(pool_size = (2, 2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, kernel_size=(3, 3),padding = 'Same',
                     activation =activ, kernel_initializer = 'glorot_uniform'))
    model.add(MaxPool2D(pool_size = (2, 2)))
    model.add(Dropout(0.25))

    model.add(Flatten())
    model.add(Dense(128, activation='relu', kernel_initializer=init))
    model.add(Dense(num_classes, activation='softmax'))
    model.summary()

    if optim == 'rmsprop':
        optimizer = RMSprop(learning_rate=lr)

    else:
        optimizer = Adam(learning_rate=lr)

    model.compile(optimizer = optimizer ,loss = "binary_crossentropy", metrics=["accuracy"])
    return model

# Set a learning rate annealer
learning_rate_reduction = ReduceLROnPlateau(monitor='val_accuracy',
                                            mode="max",
                                            patience=5,
                                            verbose=1,
                                            factor=0.5,
                                            min_lr=1e-7)
```

Figure 2: “Building the model”

Sprint #3

A simple CNN classification model was able to achieve about .7 accuracy in the test dataset.



```
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 450, 600, 64)	1,792
max_pooling2d (MaxPooling2D)	(None, 225, 300, 64)	0
dropout (Dropout)	(None, 225, 300, 64)	0
conv2d_1 (Conv2D)	(None, 225, 300, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 112, 150, 64)	0
dropout_1 (Dropout)	(None, 112, 150, 64)	0
flatten (Flatten)	(None, 1075200)	0
dense (Dense)	(None, 128)	137,625,728
dense_1 (Dense)	(None, 7)	903

```
Total params: 137,665,351 (525.15 MB)
Trainable params: 137,665,351 (525.15 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 151s 587ms/step - accuracy: 0.6659 - loss: 0.5151 - val_accuracy: 0.6522 - val_loss: 0.3342 - learning_rate: 0.0010
Epoch 2/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 103s 466ms/step - accuracy: 0.6748 - loss: 0.2173 - val_accuracy: 0.6476 - val_loss: 0.3888 - learning_rate: 0.0010
Epoch 3/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 96s 437ms/step - accuracy: 0.6843 - loss: 0.2050 - val_accuracy: 0.6782 - val_loss: 0.2790 - learning_rate: 0.0010
Epoch 4/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 142s 438ms/step - accuracy: 0.6977 - loss: 0.1929 - val_accuracy: 0.6712 - val_loss: 0.2645 - learning_rate: 0.0010
Epoch 5/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 97s 440ms/step - accuracy: 0.7268 - loss: 0.1823 - val_accuracy: 0.6323 - val_loss: 0.2532 - learning_rate: 0.0010
Epoch 6/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 146s 460ms/step - accuracy: 0.7468 - loss: 0.1670 - val_accuracy: 0.6859 - val_loss: 0.2160 - learning_rate: 0.0010
Epoch 7/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 102s 465ms/step - accuracy: 0.7750 - loss: 0.1517 - val_accuracy: 0.6489 - val_loss: 0.2421 - learning_rate: 0.0010
Epoch 8/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 137s 441ms/step - accuracy: 0.8061 - loss: 0.1355 - val_accuracy: 0.6809 - val_loss: 0.2357 - learning_rate: 0.0010
Epoch 9/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 147s 464ms/step - accuracy: 0.8368 - loss: 0.1151 - val_accuracy: 0.6872 - val_loss: 0.2627 - learning_rate: 0.0010
Epoch 10/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 102s 463ms/step - accuracy: 0.8703 - loss: 0.0992 - val_accuracy: 0.6892 - val_loss: 0.3080 - learning_rate: 0.0010
Epoch 11/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 142s 464ms/step - accuracy: 0.8909 - loss: 0.0886 - val_accuracy: 0.6872 - val_loss: 0.3660 - learning_rate: 0.0010
Epoch 12/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 142s 464ms/step - accuracy: 0.9087 - loss: 0.0740 - val_accuracy: 0.6785 - val_loss: 0.3938 - learning_rate: 0.0010
Epoch 13/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 136s 437ms/step - accuracy: 0.9304 - loss: 0.0598 - val_accuracy: 0.6762 - val_loss: 0.4344 - learning_rate: 0.0010
Epoch 14/50
220/220 ━━━━━━━━━━━━━━━━━━━━ 102s 463ms/step - accuracy: 0.9476 - loss: 0.0476 - val_accuracy: 0.6745 - val_loss: 0.4573 - learning_rate: 0.0010
Epoch 15/50
219/220 ━━━━━━━━━━━━━━━━━━━━ 0s 372ms/step - accuracy: 0.9526 - loss: 0.0447
Epoch 15: ReduceLROnPlateau reducing learning rate to 0.00050000002374872577.
220/220 ━━━━━━━━━━━━━━━━━━━━ 96s 437ms/step - accuracy: 0.9552 - loss: 0.0408 - val_accuracy: 0.6682 - val_loss: 0.5274 - learning_rate: 0.0010
Epoch 16/50
```

Figure 3: “Model training”

A RESNET50 implementation would increase this accuracy although requiring more resources for training (about 13GB of VRAM were required to train this CNN using a T4 inside a Google Colab environment using the Torch’s `DataLoader`).

Next steps: Train a RESNET50 with the dataset and implement segmentation method to separate the skin from the lesion itself. Then ship it to a AWS server and build a front-end to use the service.